

university of groningen

Performance Engineering: Lecture #12

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-10-08

Overview



1. Memory-level Parallelism
2. Memory Prefetching
3. Data Structures
4. Final

Content



1. Memory-level Parallelism

2. Memory Prefetching

3. Data Structures

4. Final

Memory-Level Parallelism: Introduction



- **Memory-level parallelism (MLP)** = Measure of the number of outstanding and overlapping memory requests a system can have at a given time
- **Recall**: Requests to the memory are generated by the cache controllers
  - **MLP measure**: Number cache misses simultaneously outstanding
- **Background**: The number of outstanding requests per cache is limited by the size of the **Miss Status Handling Registers (MSHR)**
  - Typical MSHR size is $O(10)$ (smaller for L1, larger for LLC)
- **Strategies** for enabling more MLP:
  - Larger number of cores (and caches)
  - Larger out-of-order depth to enable earlier start of load instructions
  - Larger MSHRs
  - Memory prefetching

MSHR Register



- **Role**: Keep track of
  - Outstanding cache misses
  - Pending load/store accesses that refer to the missing cache line
- **Data structure** of an MSHR
  - Valid bit
  - Cache line address (to match incoming accesses)
  - Control/status bits
  - For each pending load/store
    - Valid, type, data size, byte in cache line, destination register or store buffer
- **Actions** involving MSHR
  - On a cache miss: Search MSHR for a pending access to the same cache line
    - Found: Allocate a load/store entry in the same MSHR entry
    - Not found: Allocate a new MSHR
    - No free entry: stall
  - When a cache line returns from the next level in memory: Search and update MSHR

Memory-Level Parallelism: Little's Law



- **Reminder** Little's Law: $N = X \cdot R$
  - N: Average number of requests in the system
  - X: Throughput
  - R: Residence time
- **Consider** the following case

$$\begin{aligned} N &= N_{\text{core}} \cdot N_{\text{MSHR}} \\ N_{\text{MSHR}} &= 16 \\ R &= 100\,\text{ns} \end{aligned}$$

- **Compute** the number of cores needed to saturate a memory bandwidth of $500\,\text{GB/s}$ assuming a cache line size of $64\,\text{B}$

$$N_{\text{core}} = \frac{(500\,\text{GB/s}) \cdot (100\,\text{ns})}{(64\,\text{B}) \cdot 16} \simeq 49$$

Increasing Memory-Level Parallelism



- Performance engineering means to improve memory-level parallelism
  - Increase the number of independent load instructions
  - Use more cores (and caches), e.g. by multi-threading
  - Improve memory prefetching
    - Use memory patterns that are more suitable for prefetcher
    - Consider software memory prefetching

Content



1. Memory-level Parallelism

2. Memory Prefetching

3. Data Structures

4. Final

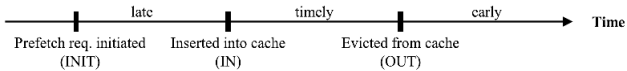
Memory Prefetchers



- **Idea**: Load data before program issues the instruction to load this data
- **Memory prefetcher** = Mechanism to fetch data from memory (or a lower cache level) to a (higher-level) cache
- **Software memory prefetching** relies on insertion of prefetch instructions into the program
- **Hardware memory prefetching** requires the following to be implemented
  - **Monitor**: Observe (and possibly record) memory access pattern
  - **Predictor**: Predict memory addresses to be accessed in the future
  - **Fetcher**: Issue a cache line fetch request
- Modern processors implement multiple prefetchers
  - Separate data and instruction prefetchers
  - Memory to cache prefetchers
  - Cache to cache prefetchers

Memory Prefetcher: Metrics

- **Metrics** to assess prefetcher implementations
  - **Coverage**: Number of misses that can be prefetched
  - **Timeliness**: Distance between prefetch and original miss
  - **Accuracy**: Probability that a prefetched line is used before being replaced



[Lee et al., 2012]

- Metric assessment is important to **minimise risk of negative performance impact**
  - Memory bus contention
  - Cache pollution

Monitoring of Memory Prefetchers

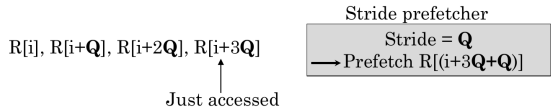


- Support for monitoring hardware prefetching is **not standardised**
- Some processors provide suitable **hardware counters**
- Example(s):
  - Kunpeng 920: The `hit_on_prf` counts the number of hits on prefetched data

Hardware Memory Prefetcher (1/2)

- Popular **memory prefetcher algorithms**
  - **Tagged prefetcher** [A.J. Smith; 1982]
    - Associates a tag bit with every memory block
    - Use tag to detect when a block is demand-fetched or a prefetched block is referenced for the first time
    - In either of these cases, the next sequential block is fetched
  - **Stride prefetcher**
    - Monitor sequence of memory addresses to update a Reference Prediction Table (RPT)
    - Use RPT to identify strided memory access patterns to predict addresses used in the future

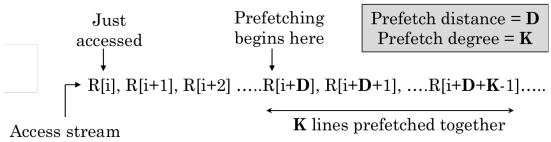
[Mittal; 2016]



Hardware Memory Prefetcher (2/2)

- **Key parameters**
  - Prefetch distance
  - Prefetch degree

[Mittal; 2016]



Software Memory Prefetching Instructions (1/2)



- Some processors support **prefetching instructions or prefetch routines**
- Options to leverage such instructions
  - Let the compiler generate instructions or calls
  - Leverage compiler-specific support
    - Example GCC: `__builtin_prefetch(const void *addr, ...)`
      - Optional argument `rw` $= 0, 1, 2$: '1' means prepare for write
      - Optional argument `locality` $= 0, 1, 2, 3$: degree of temporal locality

```
1 void vcpy(double *restrict a, double *restrict b, int n, int d)
2 {
3   for (int i = 0; i < n; i++) {
4     a[i] = a[i] + b[i];
5     __builtin_prefetch (&a[i+d], 1, 0);
6     __builtin_prefetch (&b[i+d], 0, 0);
7   }
8 }
```

Note: Data prefetch does not generate faults if the address is invalid

Software Memory Prefetching Instructions (2/2)



- Options to leverage such instructions (cont.)
  - Use of intrinsics
    - Example Intel: `_mm_prefetch(const void *, enum _mm_hint)`

Hint	Purpose
`_MM_HINT_T0`	Load data to L1D and L2 cache
`_MM_HINT_T1`	Load data to L2 cache only
`_MM_HINT_NTA`	Load data to L1D and L2 cache + set non-temporal hint

Memory Prefetcher Optimization



- Performance engineering means to leverage prefetcher techniques
  - **Hardware prefetchers**
    - Aim for sequential memory accesses that benefit from tagged prefetchers and stride prefetchers
    - Vendor information on prefetcher tends to be scarse
  - **Use software prefetching**
    - Warning: This may result in non-portable code

Content



1. Memory-level Parallelism

2. Memory Prefetching

3. Data Structures

4. Final

Data Packing



- **Consider**: Complex data structures with elements of different but small size
- **Optimization target**: Reduce size of data structures
  - Minimise memory footprint
  - Make better use of memory bandwidth
  - Optimize cache reuse
- **Implementation approach**: Minimize variable size and pack multiple variables in a single Byte

```
1 typedef struct {
2   int a;
3   int b;
4   int c;
5 } flags_up_t;
```

$\Rightarrow$

```
1 typedef struct {
2   unsigned char a:4;
3   unsigned char b:2;
4   unsigned char c:2;
5 } flags_pa_t;
```

Question: How does size change?

- **Caveat**: Computing may become more expensive due to bit operations

Data Packing



- **Consider**: Complex data structures with elements of different but small size
- **Optimization target**: Reduce size of data structures
  - Minimise memory footprint
  - Make better use of memory bandwidth
  - Optimize cache reuse
- **Implementation approach**: Minimize variable size and pack multiple variables in a single Byte

```
1 typedef struct {
2   int a;
3   int b;
4   int c;
5 } flags_up_t;
```

$\Rightarrow$

```
1 typedef struct {
2   unsigned char a:4;
3   unsigned char b:2;
4   unsigned char c:2;
5 } flags_pa_t;
```

Question: How does size change?

```
sizeof(flags_up_t) = 12
sizeof(flags_pa_t) = 1
```

- **Caveat**: Computing may become more expensive due to bit operations

Data Padding



- **Consider**: Complex data structures with elements of different size causing alignment issues
- **Optimization target**: Data structures with aligned memory addresses
- **Implementation strategy**: Add padding to improve data alignment
  - Allow for memory footprint to increase
- **Example**: Aim for 32-Byte alignment

```
1 typedef struct {
2   int    i;
3   double x;
4   char   flg;
5 } mydata_ua_t;
```

$\Rightarrow$

```
1 typedef struct {
2   int    i;
3   double x;
4   char   flg;
5   char   p[11];
6 } mydata_al_t;
```

Question: How does size change?

- **Note**:
  - Compiler performs itself padding
  - Structure size is architecture dependent

Data Padding



- **Consider**: Complex data structures with elements of different size causing alignment issues
- **Optimization target**: Data structures with aligned memory addresses
- **Implementation strategy**: Add padding to improve data alignment
  - Allow for memory footprint to increase
- **Example**: Aim for 32-Byte alignment

```
1 typedef struct {
2   int    i;
3   double x;
4   char   flg;
5 } mydata_ua_t;
```

⇒

```
1 typedef struct {
2   int    i;
3   double x;
4   char   flg;
5   char   p[11];
6 } mydata_al_t;
```

Question: How does size change?

```
sizeof(flags_ua_t) = 24
sizeof(flags_ul_t) = 32
```

- **Note**:
  - Compiler performs itself padding
  - Structure size is architecture dependent

Field Re-ordering



- **Consider**: Complex data structures with elements of different size (including small sizes)
- **Optimization target**: Reduce memory footprint
- **Example**: Aim for 32-Byte alignment

```
1 typedef struct {
2   char    flg1;
3   int     i;
4   double  x;
5   double  y;
6   char    flg2;
7 } mydata_uo_t;
```

$\Rightarrow$

```
1 typedef struct {
2   char    flg1;
3   char    flg2;
4   int     i;
5   double  x;
6   double  y;
7 } mydata_re_t;
```

Question: How does size change?

- **Note**:
  - Best to order structure elements by size
  - Structure size is architecture dependent

Field Re-ordering



- **Consider**: Complex data structures with elements of different size (including small sizes)
- **Optimization target**: Reduce memory footprint
- **Example**: Aim for 32-Byte alignment

```
1 typedef struct {
2   char    flg1;
3   int     i;
4   double  x;
5   double  y;
6   char    flg2;
7 } mydata_uo_t;
```

⇒

```
1 typedef struct {
2   char    flg1;
3   char    flg2;
4   int     i;
5   double  x;
6   double  y;
7 } mydata_re_t;
```

Question: How does size change?

```
sizeof(flags_uo_t) = 32
sizeof(flags_re_t) = 24
```

- **Note**:
  - Best to order structure elements by size
  - Structure size is architecture dependent

Containers



- There is a wide variety of ready-to-use data containers in almost any language
  - Examples for C++:
    - `std::array`
    - `std::vector`
    - `std::map`
- For containers defined in the language standard, good compiler support is likely
  - **Note**: No guarantee for optimality, but likely good compromise between performance and portability

Structure Splitting



- **Consider**: Complex data structures with elements with different usage frequency
- **Optimization target**: Keep more frequently used data in cache
- **Implementation strategy**: Split data structures
- **Example** that assumes that point coordinates are access more often:

```cpp
#include <vector>

struct Point {
  int x;
  int y;
  int z;
  /*many other fields*/
};

std::vector<Point> points;
```

$\Rightarrow$

```cpp
#include <vector>

struct PointCoords {
  int x;
  int y;
  int z;
};

struct PointInfo {
  /*many other fields*/
}

std::vector<PointCoords> pointCoords;
std::vector<PointInfo> pointInfos;
```

AoS versus SoA Data Layout (1/2)



AoS = In the **Array of Structure** data layout there is a single array with each array element holding several attributes

```cpp
#include <vector>

struct Point {
   int x;
   int y;
   int z;
};

std::vector<Point> points;
```

⇔

SoA = In the **Structure of Array** data layout there is for each attribute an array

```cpp
#include <vector>

struct Points {
   std::vector<int> x;
   std::vector<int> y;
   std::vector<int> z;
} points;
```

AoS versus SoA Data Layout (2/2)



- Benefit(s) of **SoA**
  - Helps to facilitate streaming data access
  - Good of architectures that are optimised for data stream processing (e.g., GPUs)
- Benefit(s) of **AoS**
  - Often results in higher temporal data locality
  - Good of architectures with large caches (e.g., CPUs)

Multidimensional Arrays



- **Consider**: Multidimensional arrays of simple data type (e.g., double)
- **Optimization targets**:
  - Good alignment properties
  - No memory fragmentation
  - Efficient access to single elements
- **Challenge**: C and C++ do provide only limited support for multidimensional arrays

Case Study: Multidimensional Arrays



- **Purpose** of this case study: Test different implementations of a 2-dimensional array
- **Specification** for `mdarray`: Support the following functionality
  - **Allocation** of a 2-dimensional array of doubles with size defined at run-time
  - **Accumulation**: Add to a given array the values of another array ($B \leftarrow B + A$)
  - **Free** the allocated memory

mdarray: Array of Pointers



Header file:

```
double **mda_alloc(int, int);
void mda_free(double **, int, int);
void mda_accum(double **, double **, int, int);
```

Benchmark driver:

```
#include "aop.hpp"

int main() {
  const int m = (1 << 18);
  const int n = 16;

  for (int i = 0; i < 1000; i++) {
    double **a = mda_alloc(m, n);
    double **b = mda_alloc(m, n);

    mda_accum(a, b, m, n);

    mda_free(a, m, n);
    mda_free(b, m, n);
  }
}
```

Benchmark:

```
#include <stdlib.h>
#include "aop.hpp"

double **mda_alloc(int m, int n) {
  double **a = (double **) calloc(m, sizeof(double *));
  for (int i = 0; i < m; i++)
    a[i] = (double *)calloc(n, sizeof(double));
  return a;
}

void mda_free(double **a, int m, int n) {
  for (int i = 0; i < m; i++)
    free(a[i]);
  free(a);
}

void mda_accum(double **a, double **b, int m, int n) {
  for (int i = 0; i < m; i++)
    for (int j = 0; j < n; j++)
      b[i][j] += a[i][j];
}
```

mdarray: Flattened Array



Header file:

```
1 double *mda_alloc(int, int);
2 void mda_free(double *);
3 void mda_accum(double *, double *, int, int);
```

Benchmark driver:

```
1 #include "flat.hpp"
2
3 int main() {
4   const int m = (1 << 18);
5   const int n = 16;
6
7   for (int i = 0; i < 1000; i++) {
8     double *a = mda_alloc(m, n);
9     double *b = mda_alloc(m, n);
10
11    mda_accum(a, b, m, n);
12
13    mda_free(a);
14    mda_free(b);
15  }
16 }
```

Benchmark:

```
1 #include <stdlib.h>
2 #include "flat.hpp"
3
4 double *mda_alloc(int m, int n) {
5   double *a = (double *) calloc(m*n, sizeof(double));
6   return a;
7 }
8
9 void mda_free(double *a) {
10  free(a);
11 }
12
13 void mda_accum(double *a, double *b, int m, int n) {
14   for (int i = 0; i < m; i++)
15     for (int j = 0; j < n; j++)
16       b[i*n + j] += a[i*n + j];
17 }
```

mdarray: Own C++ Class



Header file:

```
1  #include <stdlib.h>
2
3  class mda2_t {
4    private:
5      double *_a; int _m, _n;
6
7    public:
8      mda2_t(int m, int n) : _m(m), _n(n) {
9        _a = (double *) calloc(m*n, sizeof(double));
10     }
11
12     ~mda2_t() { free(_a); }
13
14     double& operator()(int i, int j) {
15       return _a[i*_n + j];
16     }
17
18     int m() const { return _m; }
19     int n() const { return _n; }
20 };
21
22 void mda_accum(mda2_t &, mda2_t &);
```

Benchmark:

```
1 #include <stdlib.h>
2 #include "class.hpp"
3
4 void mda_accum(mda2_t &b, mda2_t &a) {
5   for (int i = 0; i < b.m(); i++)
6     for (int j = 0; j < b.n(); j++)
7       b(i, j) += a(i, j);
8 }
```

Benchmark driver:

```
1  #include "class.hpp"
2
3  int main() {
4    const int m = (1 << 18);
5    const int n = 16;
6
7    for (int i = 0; i < 1000; i++) {
8      mda2_t a(m, n);
9      mda2_t b(m, n);
10
11     mda_accum(a, b);
12   }
13 }
```

`mdarray`: Kunpeng 920 Results

	N_{cycle}	N_{instr}
Array of pointers, -O3	$110.0 \cdot 10^9$	$26.7 \cdot 10^{10}$
Flattened array, -O3	$8.7 \cdot 10^9$	$1.8 \cdot 10^{10}$
C++ class, -O3	$9.1 \cdot 10^9$	$1.9 \cdot 10^{10}$

- **Compiler**: GCC 14.1.0
- **Observations**:
  - The array-of-pointers version is about $10\times$ slower and requires about $15\times$ more instructions
  - The array-of-pointers requires a very large number of instructions
    - About 64 instructions per matrix element and iteration

Multidimensional Array Access



- For very high-dimensional arrays, computing element addresses can become expensive

Case Study: Tensor Contractions



- **Purpose** of this case study: Explore the overhead of index computations
- **Specification**: Implement the following tensor contraction

$$\rho_{t,p} \leftarrow \Sigma_{t,x,y,z,c} f^{(1)}_{t,x,y,z,c,c} \cdot f^{(2)}_{t,x,y,z,c,c} \cdot \phi_p$$

with

- $t = 0, \ldots, N_t - 1$
- $x, y, z = 0, \ldots, N_s - 1$
- $c = 0, \ldots, N_c - 1$
- $p = 0, \ldots, N_p - 1$

Tensor Contractions: Original



Benchmark:

```
1  #include <stdlib.h>
2
3  void contract(double *restrict f1,
4                double *restrict f2,
5                double *restrict p,
6                double *restrict r,
7                int nt, int ns, int nc, int np) {
8    for (int it = 0; it < nt; it++) {
9      for (int ip = 0; ip < np; ip++)
10       r[it*np+ip] = 0.0;
11     for (int ix = 0; ix < ns; ix++)
12       for (int iy = 0; iy < ns; iy++)
13         for (int iz = 0; iz < ns; iz++)
14           for (int ic = 0; ic < nc; ic++) {
15             int idx = it * ns * ns * ns * nc * nc +
16                       ix * ns * ns * nc * nc +
17                            iy * ns * nc * nc +
18                                 iz * nc * nc +
19                                      ic * nc +
20                                           ic;
21             for (int ip = 0; ip < np; ip++)
22               r[it*np+ip] += f1[idx] * f2[idx] * p[ip];
23           }
24   }
25 }
```

Benchmark driver:

```
1  #include <stdlib.h>
2
3  void contract(double *restrict,
4                double *restrict,
5                double *restrict,
6                double *restrict,
7                int, int, int, int);
8
9  int main() {
10   const int nt =  8;
11   const int ns =  4;
12   const int nc =  3;
13   const int np = 40;
14   const int sd = sizeof(double);
15
16   double *f1 = (double*) calloc(nt*ns*ns*ns*nc*nc, sd);
17   double *f2 = (double*) calloc(nt*ns*ns*ns*nc*nc, sd);
18   double *p  = (double*) calloc(np, sd);
19   double *r  = (double*) calloc(nt*np, sd);
20
21   for (int i = 0; i < 100000; i++)
22     contract(f1, f2, p, r, nt, ns, nc, np);
23
24   free(f1); free(f2); free(p); free(r);
25 }
```

Tensor Contractions: Improved



Benchmark:

```
1  #include <stdlib.h>
2
3  void contract(double *restrict f1,
4                double *restrict f2,
5                double *restrict p,
6                double *restrict r,
7                int nt, int ns, int nc, int np) {
8    for (int it = 0; it < nt; it++) {
9      for (int ip = 0; ip < np; ip++)
10       r[it*np+ip] = 0.0;
11     for (int ix = 0; ix < ns; ix++)
12       for (int iy = 0; iy < ns; iy++) {
13         int idx1 = it * ns * ns * ns * nc * nc +
14                          ix * ns * ns * nc * nc +
15                               iy * ns * nc * nc;
16         for (int iz = 0; iz < ns; iz++)
17           for (int ic = 0; ic < nc; ic++) {
18             int idx = idx1 + iz * nc * nc +
19                                   ic * nc +
20                                        ic;
21             for (int ip = 0; ip < np; ip++)
22               r[it*np+ip] += f1[idx] * f2[idx] * p[ip];
23           }
24       }
25   }
26 }
```

Benchmark driver:

```
1  #include <stdlib.h>
2
3  void contract(double *restrict,
4                double *restrict,
5                double *restrict,
6                double *restrict,
7                int, int, int, int);
8
9  int main() {
10   const int nt =  8;
11   const int ns =  4;
12   const int nc =  3;
13   const int np = 40;
14   const int sd = sizeof(double);
15
16   double *f1 = (double*) calloc(nt*ns*ns*ns*nc*nc, sd);
17   double *f2 = (double*) calloc(nt*ns*ns*ns*nc*nc, sd);
18   double *p  = (double*) calloc(np, sd);
19   double *r  = (double*) calloc(nt*np, sd);
20
21   for (int i = 0; i < 100000; i++)
22     contract(f1, f2, p, r, nt, ns, nc, np);
23
24   free(f1); free(f2); free(p); free(r);
25 }
```

Tensor Contractions: Kunpeng 920 Results

	N_{cycle}	N_{instr}
Original, -O1	$4.18 \cdot 10^9$	$12.0 \cdot 10^9$
Improved, -O1	$3.74 \cdot 10^9$	$12.0 \cdot 10^9$
Original, -O3	$2.12 \cdot 10^9$	$6.3 \cdot 10^9$
Improved, -O3	$2.08 \cdot 10^9$	$6.4 \cdot 10^9$

- **Compiler**: GCC 14.1.0
- **Observations**:
  - At the O1 optimization level, the gain is about $11\,\%$
  - Significantly less gain for O3 (only about $1\,\%$)

mdspan



[D. Hollman et al.; 2019]

- **mdspan** is a multidimensional array view proposed for the C++ standard
  - First developed within Kokkos
- Pseudo-code example inspired by the tensor contraction case study:

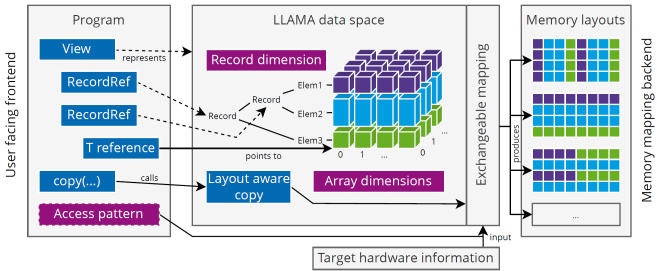
```
1  #include "mdspan.hpp"
2
3  int main() {
4    const int nt =  8;
5    const int ns =  4;
6    const int nc =  3;
7    const int np =  8;
8
9    double *_f1  = (double *) calloc(nt * ns * ns * ns * nc * nc, sizeof(double));
10
11   Kokkos::mdspan f1 { _f1, Kokkos::extents{nt, ns, ns, ns, nc, nc} };
12
13   f1(0,0,0,0,0,0) = 1.0;
14 }
```

- **Note**: Separation of **memory region** (here referenced by `_f1`) and **memory view**
- Also slicing is supported

Memory Abstractions

- **Goal**: Various efforts aim for a portable abstraction layer unifying memory optimization techniques
- Selected examples:
  - SIMD vectors: NSIMD, Highway
  - Multidimensional arrays: Kokkos
  - More general approach: LLAMA

[B. Gruber et al.; 2023]



Content



1. Memory-level Parallelism

2. Memory Prefetching

3. Data Structures

4. **Final**

Summary Lecture #12



- Memory-level parallelism and means to improve it
- Memory prefetching
- Data structures
  - Different optimization techniques
    - Data packing
    - Data padding
    - Field re-ordering
    - Use of containers
    - Structure splitting
  - Multi-dimensional arrays
  - AoS versus SoA
  - Path towards memory abstractions

Questions?



[Wiki Media]